

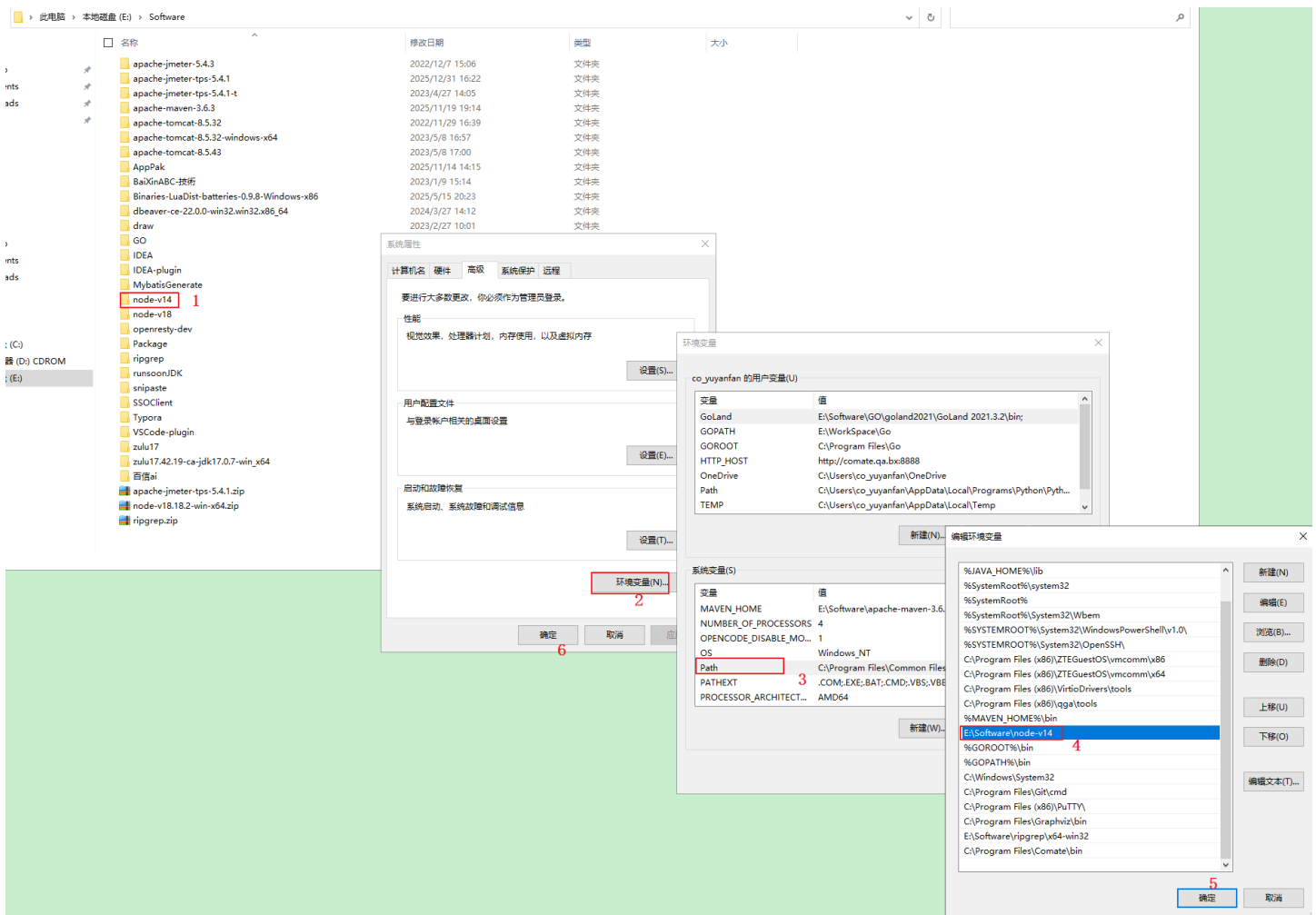
CodeGraph

1. CodeGraph CLI 安装

1、前置环境 node > 18.x

下载地址：<http://sync.qa.bx/card/api/v1/homeController/downloadFile.do?fileId=ec77b74316ca43039fc457519b52e29b>

下载解压到本地目录，配置环境变量 Win+R 输入 sysdm.cpl 回车，按图示配置即可（版本使用 v18+）



2、安装 CodeGraph

在 windows cmd 窗口执行如下命令

```
1 npm set registry http://maven.qa.bx:9090/repository/bxbank_npm-group/
2 npx @colbymchenry/codegraph@1.1.6
```

```
Microsoft Windows [版本 10.0.19045.6456]
(c) Microsoft Corporation。保留所有权利。

C:\Users\co_zhangcheng> npx @colbymchenry/codegraph@1.1.6
[#####.....] \ extract:@colbymchenry/codegraph: sill extract @colbymchenry/
```

选择在哪个Agent中安装 [如opencode或Claude Code] -> 回车

```
C:\Users\co_zhangcheng>npx @colbymchenry/codegraph
npx: installed 2 in 46.571s
T CodeGraph v1.1.6

* Which agents should CodeGraph configure?
[+] Claude Code (detected)
[ ] Cursor (not found)
[ ] Codex CLI (not found) — global only
[+] opencode (detected)
[ ] Hermes Agent (not found) — global only
[ ] Gemini CLI (not found)
[ ] Antigravity IDE (not found) — global only
[ ] Kiro (not found)
```

选择Yes -> 回车

```
* Install the codegraph CLI on your PATH? (Required so agents can launch the MCP server)
> Yes / No
```

选择All projects - 回车

```
o Installed codegraph CLI on PATH

* Apply agent configs to all your projects, or just this one?
> All projects (~/.claude, ~/.cursor, etc.)
Just this project
```

选择 Yes - 回车

```
* Auto-allow CodeGraph commands? (Skips permission prompts in Claude Code)
| > Yes / No
```

选择Yes - 回车

```
* Share anonymous usage stats? (No code, paths, or names — see TELEMETRY.md)
| > Yes / No
```

选择Yes - 回车

```
* Front-load CodeGraph on “how / where / trace” prompts? Auto-injects structural context so answers need fewer steps (adds a moment to those prompts; Claude Code only).
| > Yes / No
```

安装成功

```
* Claude Code: Updated ~/.claude.json
* Claude Code: Updated ~/.claude/settings.json
* Claude Code: Updated ~/.claude/settings.json
* Claude Code: Created ~/.claude/CLAUDE.md
* opencode: Updated ~/.config/opencode/opencode.json
* opencode: Created ~/.config/opencode/AGENTS.md
Next: index a project -----+
cd <your-project>
codegraph init          # build a project's graph (one time; auto-syncs after) |
-----+
Done! Restart your agents to use CodeGraph.
```

*** OpenCode安装验证 ***

必须验证：codegraph MCP 安装成功 及 生成 AGENTS.md，

must: 生成的 AGENTS.md文件 必须移动到agents目录下，如无agents目录，需自行创建，否则在 opencode中使用不生效 [这样opencode主Agent会优先用codegraph的图谱查询，而不是盲目翻文件]

此电脑

本地磁盘 (C:)

用户

co_zhangcheng

.config

opencode

名称	修改日期	类型	大小
agents	2026/6/30 20:09	文件夹	
node_modules	2026/4/7 16:34	文件夹	
secrets	2026/6/26 17:02	文件夹	
skills	2026/6/5 13:34	文件夹	
.gitignore	2026/2/4 16:52	文本文档	1 KB
AGENTS.md	2026/6/30 17:55	Markdown 源文件	1 KB
bun.lock	2026/4/7 16:34	LOCK 文件	2 KB
opencode.json	2026/6/30 17:55	JSON 源文件	2 KB
package.json	2026/6/16 13:57	JSON 源文件	1 KB
server-analytics.json	2026/6/26 17:00	JSON 源文件	1 KB
server-auth.json	2026/6/26 17:02	JSON 源文件	1 KB

将生成的AGENTS.md文件移动到agents目录下，如无agents目录，需自行创建，否则在opencode使用不生效

构建任何东西

E:/project/msc/msgcenter

主分支 (master)

最后修改 1秒前

1 服务器

2 MCP

LSP

插件

codegraph

image-mcp

log-mcp

审查 13

Git changes

.comate/memory.j

.gitignore

.lingma/rules/project_rule.md

pom.xml

src/main/java/com/aibank/messagecenter/A.java

.../java/com/aibank/messagecenter/controller/Animal.j...

.../com/aibank/messagecenter/controller/EarlyInitializ...

...in/java/com/aibank/messagecenter/controller/LoginController...

.../java/com/aibank/messagecenter/controller/People.j...

.../com/aibank/messagecenter/controller/TestsControll...

src/main/resources/logback-test.xml

src/main/resources/msoa.container.properties

...ibank/messagecenter/msoa/provider/LoginControlle...

*** 创建 git webhook 确保每次commit代码自动构建索引 ***

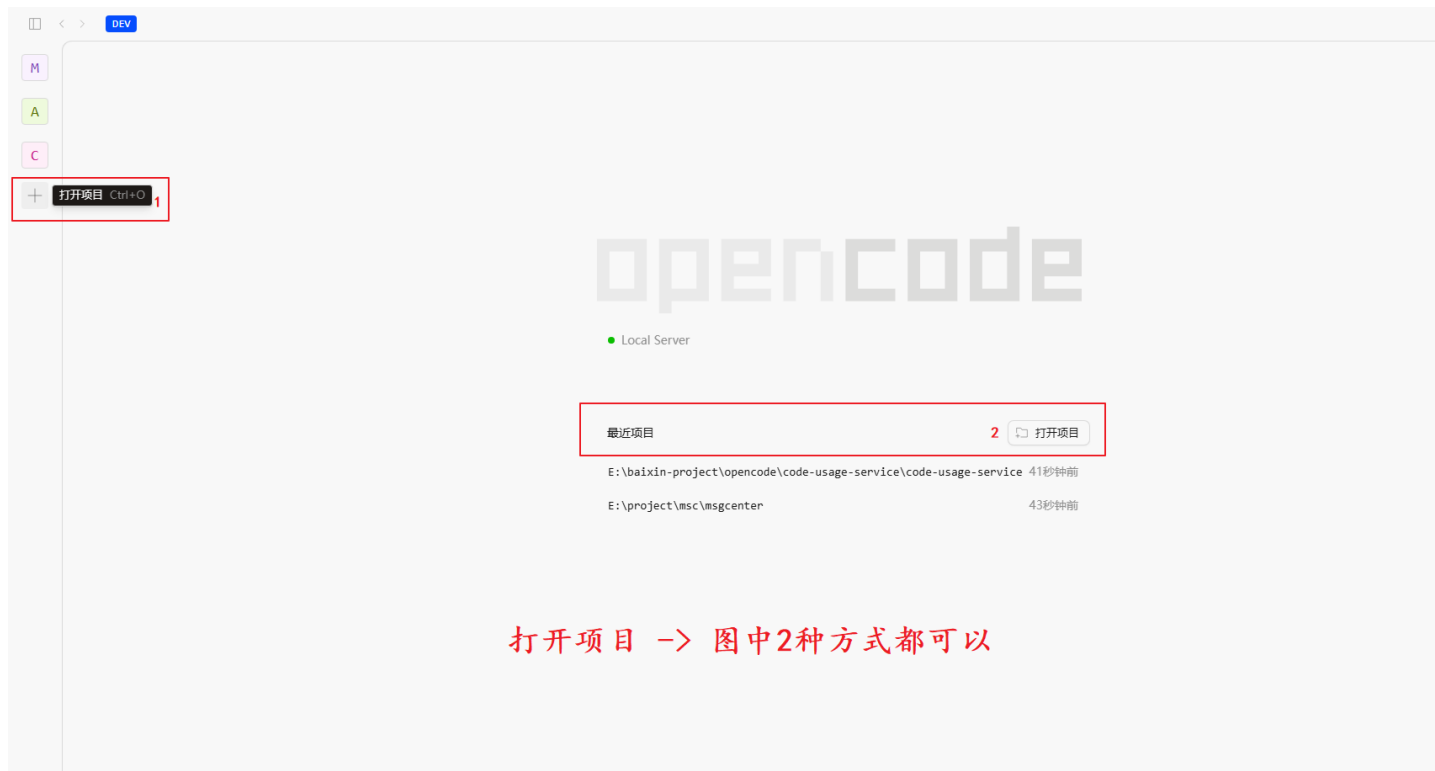
下载脚本，双击执行即可

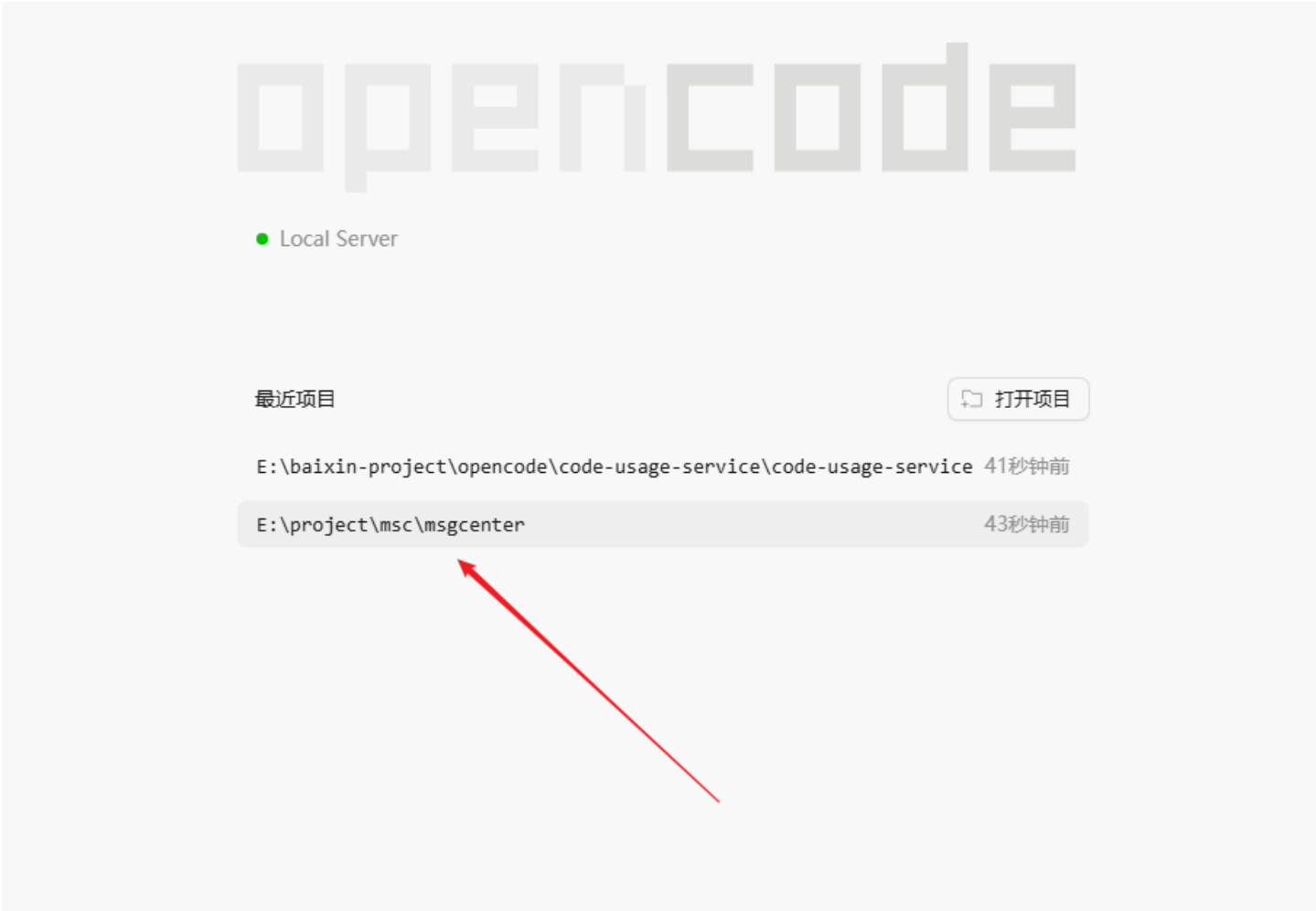
setup-git-hooks.sh	0.58 KB	2026-07-01 19:29
------------------------------------	---------	------------------

2. CodeGraph如何使用？

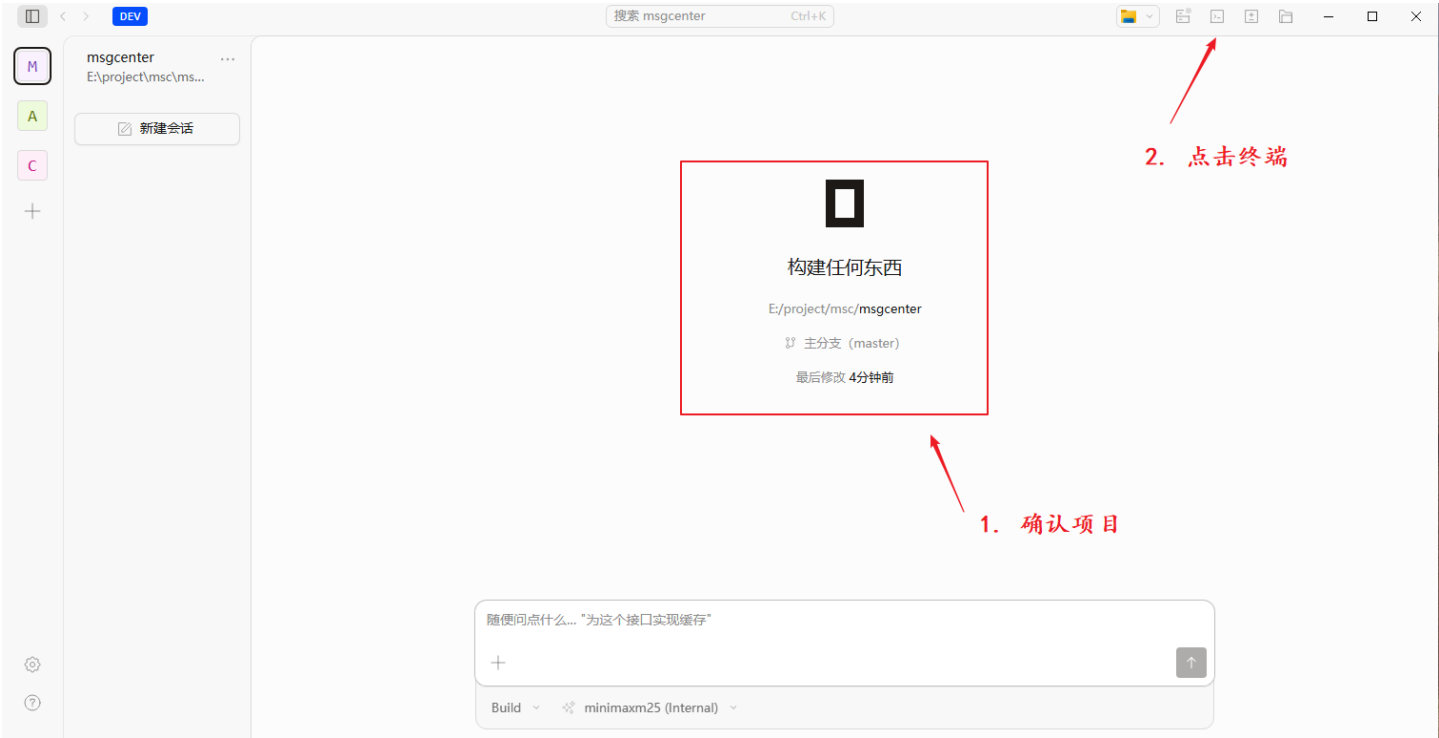
1、OpenCode中使用

1) 打开项目





2) CodeGraph 初始化项目 + 首次建图



随便问点什么... "为这个函数添加错误处理"



Build minimaxm25 (Internal)

终端 1

```
PS E:\project\msc\msgcenter> codegraph.cmd init
  Initializing CodeGraph
  ◆ Initialized in E:\project\msc\msgcenter
  |
  | * Scanning files - 321 found
  | 0 Parsing code #####----- 19%
```

*** 构建索引图谱成功 ***

终端 1

```
PS E:\project\msc\msgcenter> codegraph.cmd init
  Initializing CodeGraph
  ◆ Initialized in E:\project\msc\msgcenter
  |
  | * Scanning files - 321 found
  | * Parsing code - done
  | * Resolving refs - done
  |
  ◆ Indexed 321 files
  |
  ● 7,548 nodes, 12,224 edges in 12.5s
  |
  Done
```

3) 问答使用，分析项目

您可以在opencode问答框输入当前项目相关的问题

示例：问ldap如何工作的

查看执行过程发现调用了codegraph_codegraph_explore

If you're asking about LDAP usage in this codebase, let me know and I'll look into the project.

how does ldap work

Build · dsv4 · 19:57

Are you asking about LDAP in general, or how LDAP is used in this msgcenter codebase?

问题 1 已回答

Are you asking about LDAP in general, or how it's used in this msgcenter codebase?
In this msgcenter project

调用了 'codegraph_codegraph_explore' LDAP authentication login bind search

已探索 1 次搜索

Let me get the full LdapServiceImpl and the LoginController to see the complete flow.

已探索 2 次读取

Here's how LDAP works in this msgcenter codebase:
Two LDAP mechanisms

1. LDAP Authentication (direct bind) — LdapServiceImpl.getLogin()

随便问点什么...

+

↑

Build GLM 5.2

给出结果

file:///data/ones/files/wiki/1782907492909269.html

8/26

Understanding LDAP

Let me get the full `LdapServiceImpl` and the `LoginController` to see the complete flow.

已探索 2 次读取

Here's how LDAP works in this `msgcenter` codebase:

Two LDAP mechanisms

1. LDAP Authentication (direct bind) — `LdapServiceImpl.getIsLogin()`

`src/main/java/com/aibank/messagecenter/serviceImpl/LdapServiceImpl.java:52`

- Builds a JNDI `Hashtable` with `com.sun.jndi.ldap.LdapCtxFactory`, the LDAP URL, and simple auth.
- The security principal is `oa\<username>` (Active Directory domain prefix), credentials are the user's password.
- Attempts `new InitialLdapContext(env, null)` — If it succeeds, the password is correct; an `AuthenticationException` means wrong credentials.
- This is a direct bind approach: the user's own credentials authenticate against AD.

2. LDAP User Query — `LdapServiceImpl.getLdapUser()`

`src/main/java/com/aibank/messagecenter/serviceImpl/LdapServiceImpl.java:74`

- Uses Spring LDAP's `LdapTemplate` (configured in `LdapTemplateConfig.java:28` with a service account — a fixed admin username/password from `spring.ldap.oa.*` properties).
- Searches under `ou=中信百信银行` (queryBase) with filter `(objectClass=organizationalPerson) AND (cn=<username>)`, subtree scope.
- Returns 17 attributes: `cn`, `mail`, `memberOf`, `mobile`, `displayName`, `department`, `userAccountControl`, `whenCreated`, `lastLogon`, etc.
- If not found in the main OU, falls back to searching `ou=Supplier` and marks the user as a supplier (`setIsSupplier(true)`).
- `userAccountControl` bit & 2 determines 在职/离职 (active/resigned).
- LDAP timestamps (NT time / `yyyyMMddHHmmssZ`) are converted to Java `Date` via `getJavaDateByLdapNTTime()`.

Current State: LDAP is commented out

In `LoginController.java`, the `ldapService` is commented out (line 58-59) and the old `/loginin` endpoint that used

随便问点什么...

+

↑

Build GLM 5.2

我的分析过程分三步：

1. 用 CodeGraph 探索 LDAP 相关符号 调用 `codegraph_explore`, 查询关键词 `LDAP authentication login bind search`, 一次性拿到了 5 个相关文件的源码和调用关系：

- `LdapTemplateConfig` — LDAP 连接配置
- `LdapServiceImpl` — 认证与查询逻辑
- `SsoUserLoginService` — SSO 替代方案
- `LoginController` — 登录入口
- `LoginInVO` — 登录入参

同时拿到了 `blast radius` (谁依赖这些类), 发现 `LdapServiceImpl` 没有被调用。

2. 读取完整文件确认细节 CodeGraph 返回的 `LdapServiceImpl` 是截断的, 所以用 `Read` 工具读取了:

- `LdapServiceImpl.java` 前 50 行 — 确认 `@Value` 配置项和 `queryBase/querySupplierBase`
- `LoginController.java` 全文 368 行 — 发现 `ldapService` 被注释掉, `/loginin` 整个方法被注释, 实际走的是 `/ssologin + SsoUserLoginService`

3. 交叉验证得出结论

- `LoginController:58-59` — `@Autowired LdapServiceImpl` 被注释
- `LoginController:86-151` — 旧的 `/loginin` 方法 (调用 `ldapService.getIsLogin()`) 被注释
- `LoginController:109-110` — 注释中可见从 `ldapService` 切换到 `ssoUserLoginService` 的痕迹
- `LoginController:153` — 当前活跃的 `/ssologin` 用 `ssoUserLoginService.checkToken()`

由此得出结论: LDAP 是原始认证方式, 现已被 SSO 替代, 但代码仍保留。

2、Comate 中使用

1) IDE 打开项目

2) 初始化项目 + 首次建图

在 Terminal 中执行命令 `codegraph init`

`codegraph init`

```
PS E:\project\msc\msgcenter> cd E:\project\msc\msgcenter
```

```
PS E:\project\msc\msgcenter> pwd
```

```
Path
```

```
----
```

```
E:\project\msc\msgcenter
```

```
PS E:\project\msc\msgcenter> codegraph init
```

```
┌ Initializing CodeGraph
```

```
└◆ Initialized in E:\project\msc\msgcenter
```

```
┌ * Scanning files - 321 found
```

```
└ 0 Parsing code #####----- 65%
```

```
PS E:\project\msc\msgcenter> codegraph init
```

```
┌ Initializing CodeGraph
```

```
└◆ Initialized in E:\project\msc\msgcenter
```

```
┌ * Scanning files - 321 found
```

```
┌ * Parsing code - done
```

```
┌ * Resolving refs - done
```

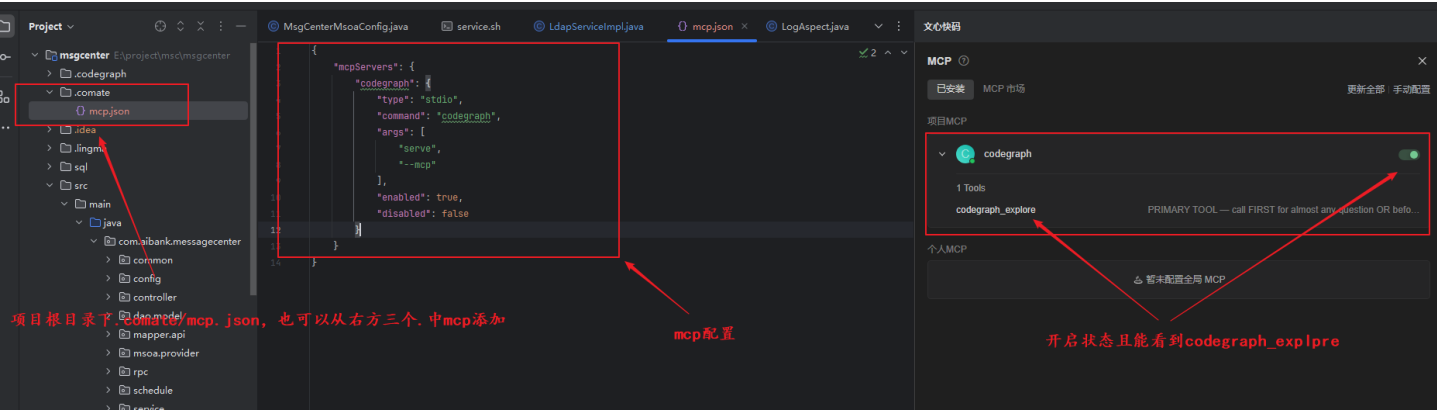
```
└◆ Indexed 321 files
```

```
● 7,548 nodes, 12,224 edges in 12.6s
```

```
└ Done
```

3) 配置 CodeGraph MCP

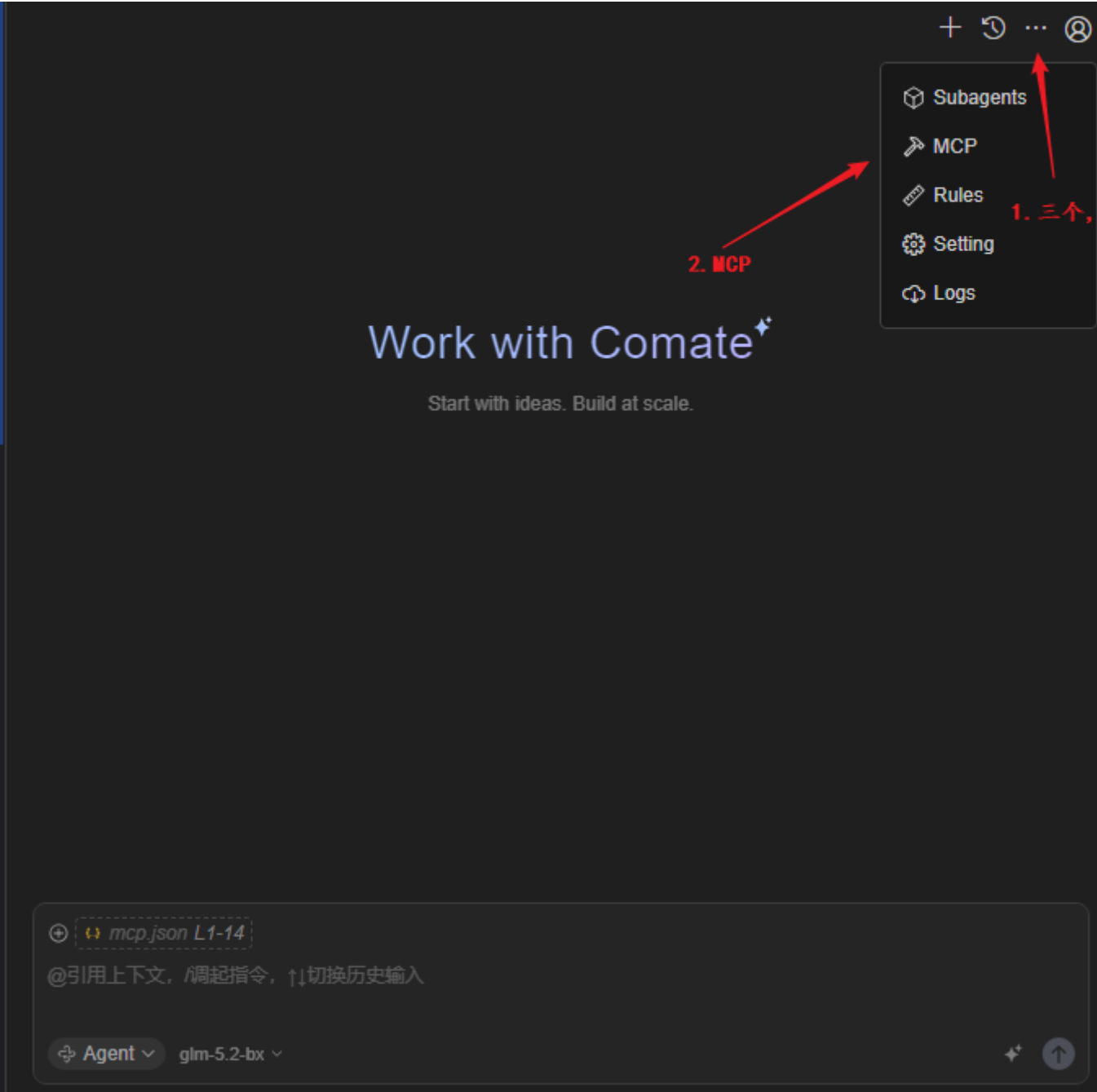
确保codegraph MCP 的状态必须为打开状态

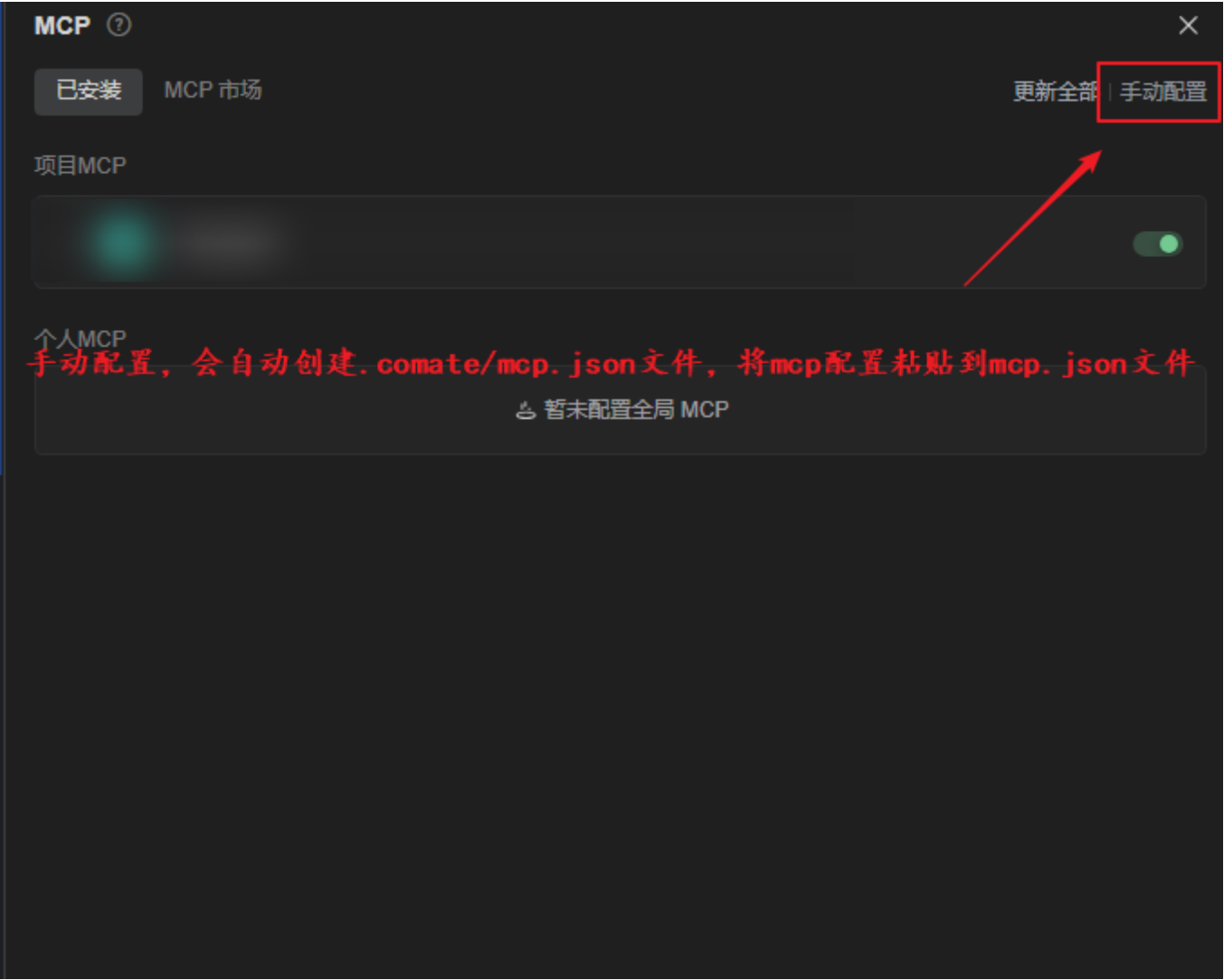


MCP配置如下：

```

1 {
2     "mcpServers": {
3         "codegraph": {
4             "type": "stdio",
5             "command": "codegraph",
6             "args": [
7                 "serve",
8                 "--mcp"
9             ],
10            "enabled": true,
11            "disabled": false
12        }
13    }
14 }
```





4) 问答使用，分析项目

您可以在comate问答框输入当前项目相关的问题

示例：调用 codegraph mcp 查询分析 how does ldap work

New Task New Task New Task **New Task**

调用codegraph mcp 查询分析 how does ldap work

⌘ Thought for a few seconds

🔗 Skill MCP: codegraph

⌘ Thought for a few seconds

👤 MCP Tool codegraph/codegraph_explore

```

100 }
106 }
...

```

> Some file sections were trimmed for size. For a specific symbol you still need, run another 'codegraph_explore' (or 'codegraph_node') with its exact name — line-numbered source, cheaper and more complete than Read.

成功

⌘ Thought for a few seconds

基于 CodeGraph 的分析，本仓库中 LDAP 功能主要由以下 4 个文件协作实现：

核心组件

1. 配置类 `LdapTemplateConfig.java` (`config/LdapTemplateConfig.java:14`)

- 通过 `@Value` 注入 `spring.ldap.oa.url`、`base`、`username`、`password` 等配置
- 定义 `@Bean oaLdapTemplate` (标记 `@Primary`、`@ConditionalOnMissingBean`)
- 内部使用 `LdapContextSource` + 匿名 `AuthenticationSource` (`getCredentials()` 返回密码、`getPrincipal()` 返回用户名) 创建 `LdapTemplate`

2. 服务实现 `LdapServiceImpl.java` (`serviceImpl/LdapServiceImpl.java`)

- 注入 `oaLdapTemplate` 以及配置项：`url`、`base`、`queryBase` (默认 `ou=中信百信银行`)、`querySupplierBase` (默认 `ou=Supplier`)
- 提供两个核心方法：
 - `nettelLogin(username, password)` (line 52) — 登录认证



@引用上下文，/调起指令，↑切换历史输入

🔗 Agent v glm-5.2-bx v



上述结果基于什么分析?

Thought for 4 seconds

上一条分析完全基于一次 `codegraph_explore` MCP 调用的返回结果，具体依据如下：

分析依据

1. 工具返回的源码 (4 个文件, verbatim 级别)

- `serviceImpl/LdapServiceImpl.java` — 含 `getIsLogin`、`getLdapUser`、`getJavaDateByLdapNTTime` 的完整实现
- `config/LdapTemplateConfig.java` — `oaLdapTemplate` Bean 定义
- `serviceImpl/vo/LdapUser.java` — POJO 字段定义
- `serviceImpl/wechat/WeixinTemplate.java` (与 LDAP 无关, 工具按相关性附带返回, 我已在末尾注明可忽略)

2. 工具返回的依赖关系 (Blast radius)

- `LdapUser` 有 2 个调用方, 均在 `LdapServiceImpl`
- `LdapTemplate` 有 1 个调用方, 在 `LdapServiceImpl`
- 两者均提示“无覆盖测试”

局限性 (我未独立验证的部分)

1. 调用链上游未追溯 — 我没有查看谁调用了 `LdapServiceImpl.getIsLogin / getLdapUser` (Controller 层、登录拦截器等), CodeGraph 这次返回里也未包含上游调用方
2. 配置实际值未读取 — `spring ldap.oa.url/base/username/password` 的真实值在 `application.yml / application.properties` 中, 我没有打开配置文件确认
3. 单次查询的覆盖范围 — CodeGraph 返回“Found 74 symbols across 4 files”, 可能存在其他与 LDAP 相关但未被本次相关性判定命中的符号 (例如 LDAP 异常处理、缓存、定时同步等)
4. 没有交叉验证 — 规则建议“对模糊或跨文件任务跑两次 Explore 交叉核对”, 本次只做了一次 MCP 调用, 未用 `grep/Read` 复核

一个客观判断

+

@引用上下文, /调起指令, ↑↓切换历史输入

Agent glm-5.2-bx

3、中心化部署API

- 1 URL :
- 2 <http://10.88.160.129:3001/api/explore>


```
3
4 请求方式： POST
5  Header： Content-Type:application/json
6 请求参数：
7  {
8      "repo": "bxbank/MSC/msgcenter",    # 代码库
9      "branch": "Release_251210_MSC",    # 分支
10     "query": "sendWeixinTemplate",      # 问题
11     "maxFiles": 6                       # 可选
12 }
13 返回参数
14 {
15     "content": [
16         {
17             "type": "text",
18             "text": "## Code Context\n\n**Query:** sendWeixinTemplate\n\n\n### Entry Points\n\n..."
19         }
20     ]
21 }
```

调用示例：

http://10.88.160.129:3001/api/explore

POST

http://10.88.160.129:3001/api/explore

Send

Save

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON (application/json)

1

2

3

4

5

6

```
{
  "repo": "bxbank/MSC/msgcenter",
  "branch": "Release_251210_MSC",
  "query": "sendWeixinTemplate",
  "maxFiles": 6
}
```

Body

Cookies

Headers (8)

Test Results

Status: 200 OK

Time: 73 ms

Size: 6.06 KB

Download

Pretty

Raw

Preview

JSON

1

2

3

4

5

```
{
  "content": [
    {
      "type": "text",
      "text": "## Code Context\n\n**Query:** sendWeixinTemplate\n\n\n### Entry Points\n\n..."
    }
  ]
}
```

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

3.使用效果

1、对比

	Token	耗时
未使用CodeGraph	>262512	11分46秒
使用CodeGraph	79326	4分58秒

1) 未使用CodeGraph

a. Token

28562 + 90285 + 56021 + 87644 + 会话压缩一次

站内模板配置与消息发送逻辑分析

帮我分析一下站内的模板配置是如何实现的？还有站内信消息发送以及微信公众号发送的逻辑

Build · dsv4 · 17:26

我将分析代码库以了解模板配置、站内信消息发送以及微信公众号消息发送逻辑。让我系统地探索一下项目。

Explore Explore template config

Explore Explore in-site message sending

Explore Explore WeChat message sending

以下是对消息中心三个核心模块的完整分析：

一、模板配置实现

1. 核心数据模型

表名	模型类	作用
template	Template.java	存储模板定义 (code、name、title、parameter JSON、channelId、source)
format_channel_template_ref	FormatChannelTemplateRef.java	三方关联表：消息格式 × 渠道 × 模板，含 effectiveTime
msg_format	MsgFormat.java	消息格式配置 (业务分类、来源、状态)
channel	Channel.java	渠道定义 (APP1/2/3、公众号)，含

审查 18 上下文 × +

会话

站内模板配置与消息发送逻辑分析

提供商

Internal OpenAI-Compatible

上下文限制

131,072

使用率

22%

输出 token

2,621

缓存 token (读/写)

0 / 0

助手消息

2

创建时间

2026年7月1日 17:26

消息数

3

模型

GLM 5.2

总 token

28,562

输入 token

25,941

推理 token

0

用户消息

1

总成本

US\$0.00

最后活动

2026年7月1日 17:36

上下文拆分

● 用户 0% ● 助手 7% ● 工具调用 50.9% ● 其他 42.1%

原始消息

user • msg_f1d00a0610017ZsiIMDd4JlHHU6

2026年7月1日 17:26

assistant • msg_f1d00a0da001HZbhSllodt1Pqy

2026年7月1日 17:26

assistant • msg_f1d09e7970012KX3sUja7LGQ1R

2026年7月1日 17:36

站内模板配置与消息发送逻辑分析 / Explore template config

Explore the codebase at E:\project\msc\msgcenter to understand how template configuration (模板配置) is implemented.

Look for:

1. Template-related entity/model classes (e.g., Template, MessageTemplate, etc.)

2. Template configuration controllers/APIs

3. Template service classes and their business logic

4. Template types and how they are categorized

5. Database tables/mappings related to templates

6. How templates are loaded and used

Search broadly using patterns like *Template*, *template*, 模板, etc. Read the key files you find.

Please provide a comprehensive summary with file paths and line numbers for all key components found. Be very thorough - check controllers, services, entities, mappers, and configuration files.

Explore - dsv4 - 17:26

I'll start by exploring the codebase structure and searching for template-related files broadly.

Shell List root directory contents

已探索 3 次搜索

It seems glob and grep tools are having issues with ripgrep download. Let me use bash commands instead to search the codebase.

Shell Find all Template-related files

Shell Find all template-related files (lowercase)

Shell List SQL files

Excellent! I found many template-related files. Let me also search for files containing "模板" (Chinese for template) and explore the overall project structure.

Shell Find files containing 模板 (Chinese for template)

Subagent sessions cannot be promoted. Back to main session.

审查 18

上下文

会话	消息数
Explore template config (@explore subagent)	29
提供商	模型
Internal OpenAI-Compatible	GLM 5.2
上下文限制	总 token
131,072	90,285
使用率	输入 token
69%	83,790
输出 token	推理 token
6,495	0
缓存 token (读/写)	用户消息
0 / 0	1
助手消息	总成本
28	US\$0.00
创建时间	最后活动
2026年7月1日 17:26	2026年7月1日 17:32

上下文拆分

用户 0.2%

助手 7.9%

工具调用 82.7%

其他 9.2%

原始消息

user • msg_f1d00bc1800195NxFtTfZjbEwB	2026年7月1日 17:26
assistant • msg_f1d00bc2c001YSlspJCFe8hr4Q	2026年7月1日 17:26
assistant • msg_f1d00dadf001NhYSitgxAKJleD	2026年7月1日 17:27
assistant • msg_f1d011110001UOSXlvVCpTVm7y	2026年7月1日 17:27
assistant • msg_f1d014539001Mo6SNlxPQsHlq2	2026年7月1日 17:27
assistant • msg_f1d0166df001ia2bpHMTNjclbx	2026年7月1日 17:27
assistant • msg_f1d01af2e0013lBzcvoqJqE8BN	2026年7月1日 17:27
assistant • msg_f1d01e390001EYhbcjMizOwd50	2026年7月1日 17:28
assistant • msg_f1d022ae50013H7foGkkHIZsQU	2026年7月1日 17:28

中间会话压缩一次

站内模板配置与消息发送逻辑分析 / Explore in-site message sending

Explore the codebase at E:\project\msc\msgcenter to understand how in-site messages (站内信) are sent.

Look for:

1. In-site message related classes (search for patterns like *SiteMessage*, *站内信*, *InSite*, *InternalMessage*, *Notice*, etc.)

2. Message sending controllers/APIs

3. Message sending service classes and their business logic

4. How messages are dispatched/routed

5. Message queue or async sending mechanisms

6. Database tables/mappings related to in-site messages

Search broadly. Read the key files you find.

Please provide a comprehensive summary with file paths and line numbers for all key components found. Be very thorough.

Explore - dsv4 - 17:26

I'll start by exploring the codebase structure and searching for in-site message related components in parallel.

Shell Check root dir and codegraph presence

已探索 4 次搜索

The glob tool requires ripgrep which can't be downloaded. Let me use bash commands (PowerShell) to explore the codebase instead.

Shell List root directory contents

Shell Find all Java and config files

This is a message center (消息中心) application. Now let me search for in-site message related content across the codebase using PowerShell's Select-String.

Shell Search for in-site message keywords

Shell Search for SysMessage references

审查 18

上下文

会话	消息数
Explore in-site message sending (@explore subagent)	28
提供商	模型
Internal OpenAI-Compatible	GLM 5.2
上下文限制	总 token
131,072	56,021
使用率	输入 token
43%	52,625
输出 token	推理 token
3,396	0
缓存 token (读/写)	用户消息
0 / 0	3
助手消息	总成本
25	US\$0.00
创建时间	最后活动
2026年7月1日 17:26	2026年7月1日 17:34

上下文拆分

用户 0.1%

助手 4.6%

工具调用 95.3%

其他 0%

原始消息

user • msg_f1d00cb44001IXp6R7yZCQg1A1	2026年7月1日 17:26
assistant • msg_f1d00cb64001GN8aUz4ktDv7sH	2026年7月1日 17:26
assistant • msg_f1d00f2c80018yUNqTdG0NgWwZ	2026年7月1日 17:27
assistant • msg_f1d0122c30016GGK893llr6zW1	2026年7月1日 17:27
assistant • msg_f1d01855e0019e9VBZ9T8Tyl4Y	2026年7月1日 17:27
assistant • msg_f1d01be8b0019PN8qiiBLK22c	2026年7月1日 17:28
assistant • msg_f1d01f21a001eEaWKMAAsZyWV6	2026年7月1日 17:28
assistant • msg_f1d021a52001FuvfY13ikKevFY3	2026年7月1日 17:28

站内模板配置与消息发送逻辑分析 / Explore in-site message sending

remaining mode/mapper files.

已探索 4 次读取

Let me now read the remaining key files - the SendStatusEnum, the ManualMsgTaskConfService interface and its implementation, and the remaining mapper files.

已探索 4 次读取

会话已压缩

Goal

- Explore the codebase at `E:\project\msc\msgcenter` to understand how in-site messages (站内信) are sent, including classes, APIs, services, dispatch/routing, async mechanisms, and database mappings.

Constraints & Preferences

- User requested comprehensive summary with file paths and line numbers for all key components
- Search broadly for patterns: `*SiteMessage*`, `*站内信*`, `*InSite*`, `*InternalMessage*`, `*Notice*`, etc.
- Be very thorough

Progress

审查 18

上下文

会话	Explore in-site message sending (@explore subagent)	消息数	28
提供商	Internal OpenAI-Compatible	模型	GLM 5.2
上下文限制	131,072	总 token	56,021
使用率	43%	输入 token	52,625
输出 token	3,396	推理 token	0
缓存 token (读/写)	0 / 0	用户消息	3
助手消息	25	总成本	US\$0.00
创建时间	2026年7月1日 17:26	最后活动	2026年7月1日 17:34

上下文拆分

站内模板配置与消息发送逻辑分析 / Explore WeChat message sending

Explore the codebase at `E:\project\msc\msgcenter` to understand how WeChat official account messages (微信公众号消息) are sent.

Look for:

- 1. WeChat-related classes (search for patterns like `*WeChat*`, `*Wechat*`, `*Wx*`, `*微信*`, `*GZH*`, `*OfficialAccount*`, `*Mp*`, etc.)
- 2. WeChat message sending controllers/APIs
- 3. WeChat message sending service classes and their business logic
- 4. WeChat API integration (template messages, subscription messages, etc.)
- 5. WeChat access token management
- 6. Database tables/mappings related to WeChat messages

Search broadly. Read the key files you find.

Please provide a comprehensive summary with file paths and line numbers for all key components found. Be very thorough.

Explore - dsv4 - 17:27

I'll start by exploring the codebase structure and searching for WeChat-related components broadly.

已探索 1 次读取, 3 次搜索

Glob seems to have issues. Let me try using bash to find files instead.

审查 18

上下文

会话	Explore WeChat message sending (@explore subagent)	消息数	16
提供商	Internal OpenAI-Compatible	模型	GLM 5.2
上下文限制	131,072	总 token	87,664
使用率	67%	输入 token	83,420
输出 token	4,244	推理 token	0
缓存 token (读/写)	0 / 0	用户消息	1
助手消息	15	总成本	US\$0.00
创建时间	2026年7月1日 17:27	最后活动	2026年7月1日 17:30

上下文拆分

● 用户 0.2%

● 助手 5.3%

● 工具调用 80.8%

● 其他 13.7%

b. 问答耗时：

11分46秒

站内模板配置与消息发送逻辑分析

整体架构图

消息模板配置体系

template + format_channel_ref
+ msg_format + channel

(每分钟缓存刷新)

v

MessageConfigCache (内存缓存)
MsgFormatCache
└ ChannelCache
└ TemplateCache

站内信 (APP)
渠道: 1100/
1200/1300

公众号 (WX)
渠道: 2100

直接更新DB
send_status=2
客户端按需查询

getOpenId(RPC)
→ sendWeixin
Template()
→ 微信API

如需深入某个具体环节 (如模板参数的JS表达式转换、定时任务调度细节、人工消息客户群选择逻辑等), 可以进一步分析。

Build · dsv4 · 11分 46秒

审查 18

上下文

会话	消息数
站内模板配置与消息发送逻辑分析	3
提供商	模型
Internal OpenAI-Compatible	GLM 5.2
上下文限制	总 token
131,072	28,562
使用率	输入 token
22%	25,941
输出 token	推理 token
2,621	0
缓存 token (读/写)	用户消息
0 / 0	1
助手消息	总成本
2	US\$0.00
创建时间	最后活动
2026年7月1日 17:26	2026年7月1日 17:36

上下文拆分

用户 0%

助手 7%

工具调用 50.9%

其他 42.1%

原始消息

user • msg_f1d00a0610017ZsiMDd4JjHHU62026年

assistant • msg_f1d00a0da001HZbhSllodt1Pqy2026年

assistant • msg_f1d09e7970012KX3sUja7LGQ1R2026年

海量搜索文件

Shell Find all Java and config files

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetManualMsgTaskConfListOutVO.java

a

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetMsgFormatByBizClassInVo.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetMsgFormatByBizClassOutVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetMsgFormatListInVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetMsgFormatListOutVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetSelectedChannelListInVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetSelectedChannelListOutVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetTemplateListDetailInVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetTemplateListDetailOutVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetTemplateListInVO.java

E:\project\msc\msgcenter\src\main\java\com\airbank\messagecenter\controller\vo\GetTemplateListOutVO.java

This is a message center (消息中心) application. Now let me search for in-site message related content across the codebase using PowerShell's Select-String.

- 2) 使用CodeGraph
- a. Token

Token：79326

模板配置与消息发送逻辑分析

帮我分析一下站内的模板配置是如何实现的？还有站内信消息发送以及微信公众

号发送的逻辑

我来分析一下站内模板配置、站内信消息发送以及微信公众发送的逻辑。先探索一下代码结构。

调用了 'codegraph_codegraph_explore' 站内信模板配置 消息发送 微信公众发送 tem... projectPath=E:\proje... maxF...

已探索 1 次搜索

让我用其他方式来探索代码结构。

Shell List all Java files in the project

已探索 1 次读取

读取 msgcenter

现在我对项目结构有了整体了解。让我深入阅读关键文件。

已探索 4 次读取

读取 TemplateController.java

读取 TemplateService.java

读取 TemplateServiceImpl.java

读取 Template.java

现在让我读取消息发送和微信公众号相关的核心文件。

已探索 4 次读取

读取 MessageServiceImpl.java

读取 MsgServiceImpl.java

读取 WeixinServiceImpl.java

随便问点什么...

Build GLM 5.2

审查 18 上下文

会话	消息数
模板配置与消息发送逻辑分析	12
提供商	模型
Internal OpenAI-Compatible	GLM 5.2
上下文限制	总 token
131,072	79,326
使用率	输入 token
61%	75,471
输出 token	推理 token
3,855	0
缓存 token (读/写)	用户消息
0 / 0	1
助手消息	总成本
11	US\$0.00
创建时间	最后活动
2026年7月1日 17:46	2026年7月1日 17:48

上下文拆分

● 用户 0%

● 助手 3.7%

● 工具调用 76.3%

● 其他 20%

原始消息

user • msg_f1d128e1400145L8FsktdYJV2k	2026年7月1日 17:46
assistant • msg_f1d128f2d001o6WpX6R7hgEd9d	2026年7月1日 17:46
assistant • msg_f1d12be390016OgSBar1bCpKsS	2026年7月1日 17:46
assistant • msg_f1d12daa900128ek9Pwlz1FqUq	2026年7月1日 17:46
assistant • msg_f1d1307f9001qm9WgeoleLnpZx	2026年7月1日 17:46
assistant • msg_f1d132696001RTZ4fgMb57R0eG	2026年7月1日 17:47
assistant • msg_f1d1368d8001fBS3Uqw0TpS3Pf	2026年7月1日 17:47
assistant • msg_f1d13905c001N0sXF3AZFK0SDe	2026年7月1日 17:47
assistant • msg_f1d13be18001ZXRIJts01rsAJM	2026年7月1日 17:47

b. 问答耗时：

4分58秒

模板配置与消息发送逻辑分析

金融网关 / 人工消息定时任务

MsgServiceImpl.sendCommon()

参数校验+转换
(模板引擎)

消息存储
(DB持久化)

发送执行
(异步线程池)

TemplateParameterVO

├ 输入参数校验

├ 输出参数转换

└ 模板渲染

Message

"APP"

"公众号"

+Channel

写库即发送

WeixinServiceImpl

├ getOpenId()

├ flushToken()

└ sendWeixinTemplate()

SysMessage

(全体消息)

searchInnerMessage

(延迟发送)

ScheduleMsgTrigger

(定时发送)

关键设计特点:

1. 模板与渠道解耦: 一个 MsgFormat 可关联多个渠道, 每个渠道绑定不同 Template

2. 参数转换引擎: 支持输入校验 + JS脚本转换 + 模板渲染的三层处理

3. 全体消息延迟发送: SysMessage 不直接发送, 用户查询时才生成个人消息

4. 多渠道统一入口: sendCommon 统一处理, 按 parentChannelName 分发到不同渠道

Build · dsv4 · 4分 58秒

随便问点什么...

+

Build GLM 5.2

审查 18

上下文

会话

模板配置与消息发送逻辑分析

提供商

Internal OpenAI-Compatible

上下文限制

131,072

使用率

61%

输出 token

3,855

缓存 token (读/写)

0 / 0

助手消息

11

创建时间

2026年7月1日 17:46

消息数

12

模型

GLM 5.2

总 token

79,326

输入 token

75,471

推理 token

0

用户消息

1

总成本

US\$0.00

最后活动

2026年7月1日 17:48

上下文拆分

用户 0%

助手 3.7%

工具调用 76.3%

其他 20%

原始消息

user • msg_f1d128e1400145L8FsktdYJV2k

assistant • msg_f1d128f2d001o6WpX6R7hgEd9d

assistant • msg_f1d12be390016OgSBar1bCpKsS

assistant • msg_f1d12daa900128ek9Pwlz1FqUq

assistant • msg_f1d1307f9001qm9WgeoleLnpZx

assistant • msg_f1d132696001RTZ4fgMb57R0eG

assistant • msg_f1d1368d8001fBS3Uqw0TpS3Pf

assistant • msg_f1d13905c001N0sXF3AZFK05De

assistant • msg_f1d13be18001ZXRJlts01rsAJM

精准搜索文件

现在我对项目结构有了整体了解。让我深入阅读关键文件。

已探索 4 次读取

读取 TemplateController.java

读取 TemplateService.java

读取 TemplateServiceImpl.java

读取 Template.java

现在让我读取消息发送和微信公众号相关的核心文件。

已探索 4 次读取

读取 MessageServiceImpl.java

读取 MsgServiceImpl.java

读取 WeixinServiceImpl.java

读取 WxService.java

file:///data/ones/files/wiki/1782907492909269.html

23/26

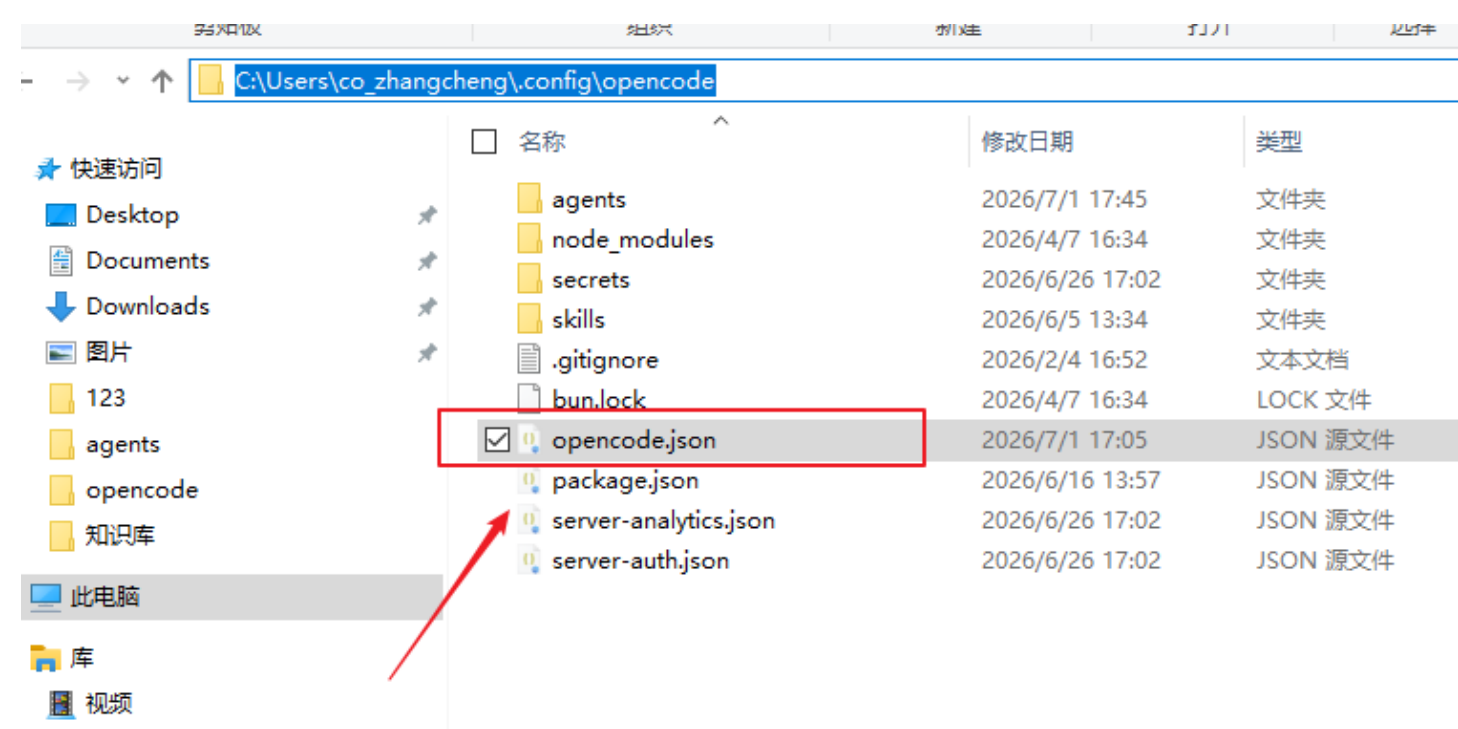
4、Q & A

1、codegraph init

codegraph init 创建 .codegraph/目录并构建完整图谱，只需一条命令，搞定之后，原生文件监视器慧在每次更改时保持索引同步，因为几乎需要手动重建，如果您确实需要，按如下执行，每个项目的数据存储都存储在 .codegraph/ 项目根目录下的一个目录中，其中包含 SQLite 数据库（codegraph.db ），所有数据都不会离开您的计算机

2、opencode 无 codegraph mcp

需检查 opencode.json 文件中是否有 codegraph mcp 配置，配置文件路径如下：



检查如下配置

```

    },
    "codegraph": {
      "type": "local",
      "command": [
        "codegraph",
        "serve",
        "--mcp"
      ],
      "enabled": true
    }
  },
  "opencode": {
    "enabled": true
  }
}
```


3、 全量重建索引

codegraph index # full re-index

```
PS E:\project\msc\msgcenter> codegraph index
┌ Indexing project
│
│  * Scanning files - 321 found
│  * Parsing code #####----- 30%
└
```

4、 只更新变化的部分索引

codegraph sync # incremental update of changed files

```
PS E:\project\msc\msgcenter> codegraph sync
┌ Syncing CodeGraph
│
│  * Scanning files
│  * Parsing code - done
│  * Resolving refs - done
│
│  ◆ Synced 1 changed files
│
│  ● Modified: 1 - 43 nodes in 2.6s
└ Done
```

5、 codegraph status 用来查看当前项目索引的健康状态和统计信息

执行后会告诉你：

索引了多少个文件

索引了多少个符号

有多少条边（调用关系、依赖关系等）

索引是否正常、有没有损坏

上次同步的时间

加 --json 参数可以输出机器可读的JSON格式，方便在脚本或CI里用，codegraph status --json典型使用场景：跑sync或explore之前先status看一眼，确认索引事完整的、没有落后太多、如果状态异常（比如节点

数为0或文件数明显不对），就该使用codegraph index --force重建索引了。

```
PS E:\project\msc\msgcenter> codegraph status
```

CodeGraph Status

Project: E:\project\msc\msgcenter

Index Statistics:

Files:	321
Nodes:	7,548
Edges:	12,224
DB Size:	15.36 MB
Backend:	node:sqlite - built-in (full WAL)
Journal:	wal

Nodes by Kind:

method	2,830
import	2,169
field	1,188
constant	315
file	314
namespace	293
class	252
route	58
enum_member	57
interface	53
enum	19

Files by Language:

java	295
xml	19
properties	6
yaml	1

```
PS E:\project\msc\msgcenter>
PS E:\project\msc\msgcenter>
PS E:\project\msc\msgcenter>
PS E:\project\msc\msgcenter> codegraph status --json
{"initialized":true,"version":"1.1.6","projectPath":"E:\\project\\msc\\msgcenter","indexPath":"E:\\project\\msc\\msgcenter\\.codegraph","lastIndexed":"2026-06-30T11:41:56.863Z","fileCount":321,"nodeCount":7548,"edgeCount":12224,"dbSizeBytes":16185472,"backend":"node-sqlite","journalMode":"wal","nodesByKind":{"class":252,"constant":315,"enum":19,"enum_member":57,"field":1188,"file":314,"import":2169,"interface":53,"method":2830,"namespace":293,"route":58,"Languages":["java","properties","xml","yaml"],"pendingChange":{},"added":0,"modified":0,"removed":0,"worktreeMismatch":null,"index":{"builtWithVersion":"1.1.6","builtWithExtractionVersion":24,"currentExtractionVersion":24,"reindexRecommended":false}}
PS E:\project\msc\msgcenter>
```